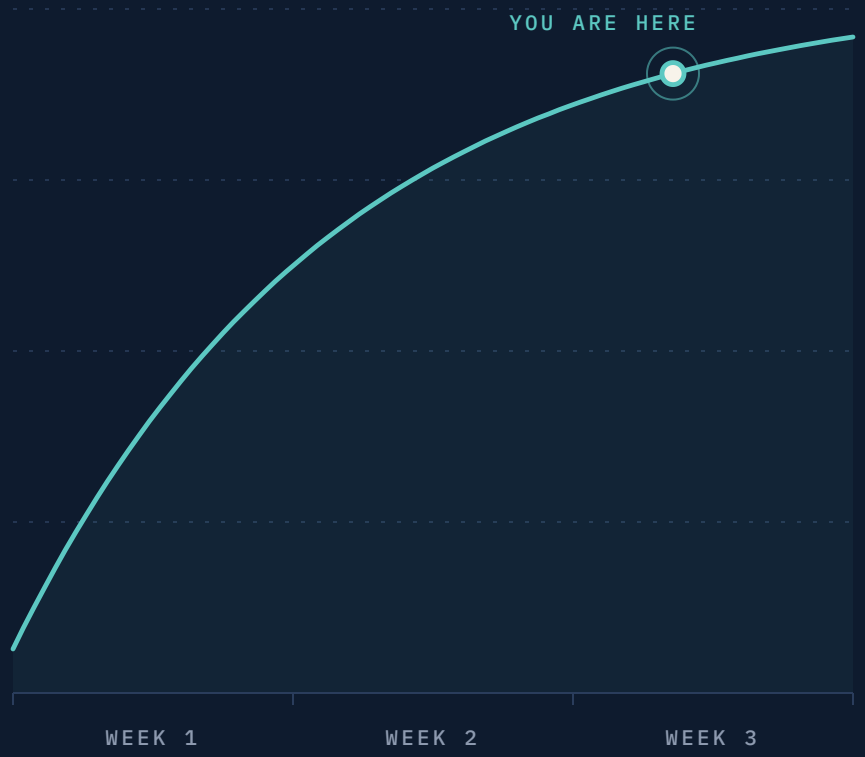
WEEK 3 · DAY 2 · OVERVIEW

Feature Engineering on Financial Ratios



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 10 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 11 left two models on the Polish training rows: LightGBM at its defaults, at a fold average precision of 0.7783, and Course 4's logistic pipeline, at 0.6624.

The colleague who brought the file asks what every analyst asks next: we build ratios by hand, so can better columns lift either model?

Feature engineering is building new columns from the ones a file already has: a cap, a ratio, a count, a transform, a share.

A warm-up comes first, on the course's synthetic bonds, where the answer is known because the generator's recipe is.

Then the Polish companies, where nobody hands us a recipe.

A fitted spread here describes synthetic bonds, and a model's output on the Polish file is the model's probability of bankruptcy in this file: neither is a credit view on any bond or company.

OBJECTIVES

Write a known shape into a column and score it on folds, build a ratio from two ratios and guard its division, and turn gaps into columns.

Transform a long-tailed ratio for a linear model, learn a column's numbers inside each fold, encode a group's bankrupt share inside the folds, and weigh what each new column adds against the fold noise.

ABOUT 45 MIN WITH EXERCISES / 10 SECTIONS / 4 EXERCISES

Your Route Through Day 2

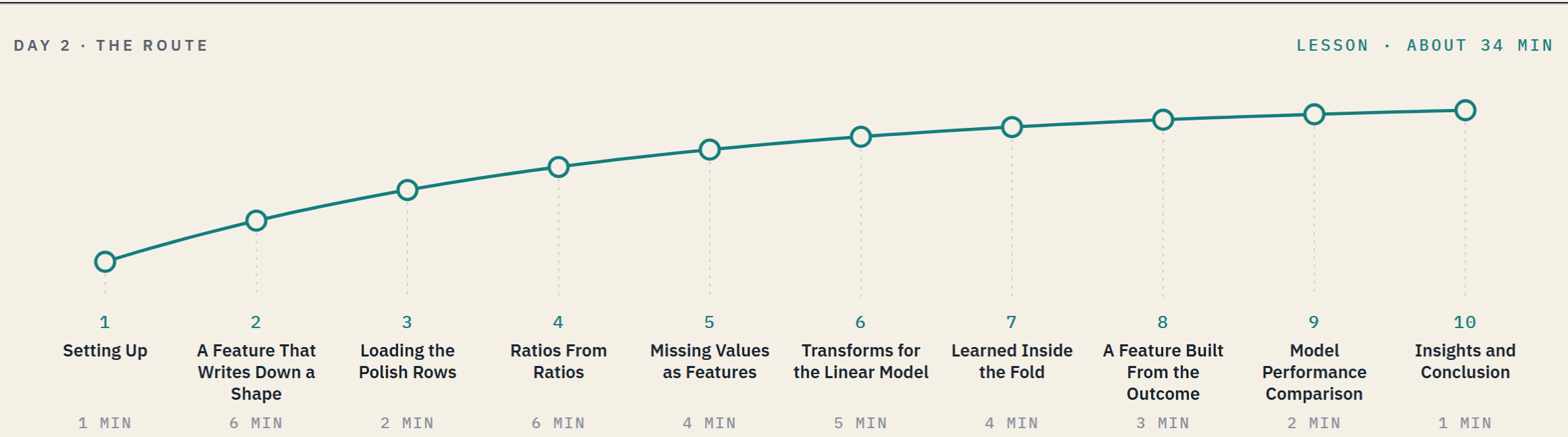


FIGURE 1 Ten sections, in order, with about how long each takes.

Time. About 34 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
```

THE NEW PIECES

```
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingRegressor
from lightgbm import LGBMClassifier
```

THE RATIOS THIS NOTEBOOK NAMES

- **`rating`** and its **`score`**, 1 for AAA up to 18 for CCC
- **`maturity`** and **`as_of_date`**, which give years to maturity
- **`oas`**: option-adjusted spread, bp; **the target is its log**
- **`Attr1`** to **`Attr64`**: the page's X1 to X64, ratios from the statements

Run this cell and move on: it fixes the seed and finds the course data.

A Feature That Writes Down a Shape

THE BONDS AND COURSE 4'S SPLIT

```
train, test = train_test_split(bonds, test_size=0.25, random_state=SEED)
```

```
Training bonds: 615  Past 10 years: 181
```

DAY 7'S TWO NUMBERS

```
line, years       42.3147
boosting, years   41.2423
dtype: float64
```

A CAP IN ONE COLUMN, THEN THE LINE SCORED ON IT

```
train["years_capped"] = train["years_to_maturity"].clip(upper=10)
train["log_years"] = np.log(train["years_to_maturity"])
bond_mae["line, log years"] = fold_bp(LinearRegression(), ["score", "log_years"])
bond_mae["line, capped"] = fold_bp(LinearRegression(), ["score", "years_capped"])
```

THE QUESTION

Q. Give the line years capped at 10 in place of years, and nothing else. Write down its fold MAE in bp, against 42.31 for the line and 41.24 for boosting.

Boosting finds the bend at 10 years with its splits; one straight slope in years cannot bend.

Loading the Polish Rows

COURSE 4'S ROWS

```
bankrupt = pd.read_csv(DATA + "polish_bankruptcy_5year.csv")
unique = bankrupt.drop_duplicates()
```

```
5910 read, 5850 unique.  Training: 4680 rows, 326 bankrupt, 6.97 percent
```

THE TWO MODELS TO BEAT

```
booster = LGBMClassifier(random_state=SEED, n_jobs=1, verbose=-1)
```

	1	2	3	4	5	MEAN	SD
LightGBM	0.7077	0.7988	0.7434	0.8013	0.8405	0.7783	0.0469
logistic	0.5381	0.7195	0.6186	0.6750	0.7607	0.6624	0.0780

Every new column today is scored against these two means, on these five folds, beside fold sds of 0.0469 and 0.0780.

Ratios From Ratios

PLAIN DIVISION

```
plain = X_train["Attr7"] / X_train["Attr2"]
```

```
Attr2 at 0: 12  Attr10 at 0: 1
EBIT to liabilities, infinite: 9  Missing: 5
```

THE GUARD

```
guarded = plain.replace([np.inf, -np.inf], np.nan)
```

```
Infinite: 0  Missing: 14
```

THREE RATIOS, EVERY DIVISION GUARDED

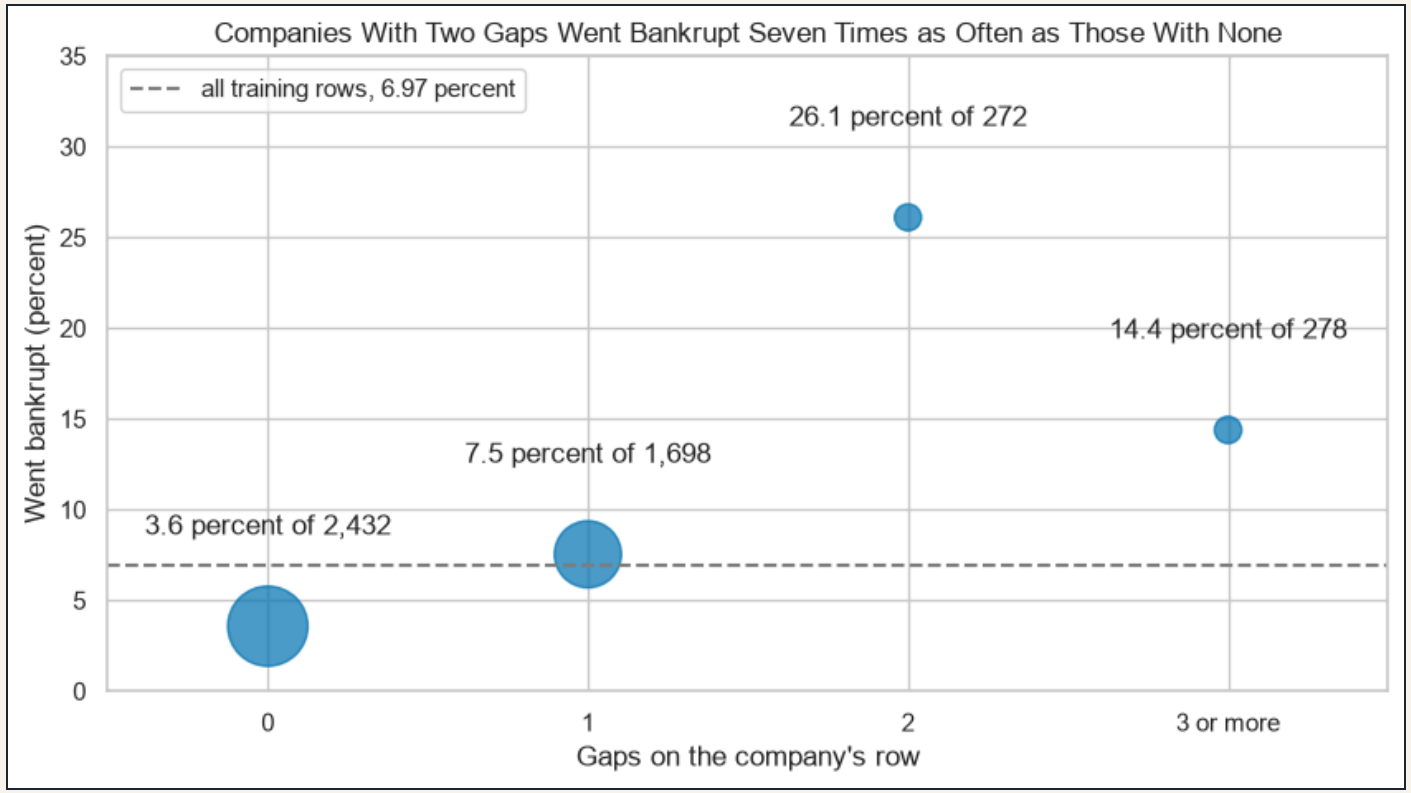
```
def add_ratios(frame):
    ratios = pd.DataFrame({"debt_to_equity": frame["Attr2"] / frame["Attr10"],
                           "ebit_to_liabilities": frame["Attr7"] / frame["Attr2"],
                           "short_term_share": frame["Attr51"] / frame["Attr2"]})
    return pd.concat([frame, ratios.replace([np.inf, -np.inf], np.nan)], axis=1)
ratio_step = FunctionTransformer(add_ratios)
```

THE QUESTION

Q. Sketch debt to equity, **Attr2 / Attr10**, against **Attr2** from 0 to 2. How does it move as **Attr2** nears 1, and past it?

12 companies report no liabilities at all, and plain division gives 9 of them an infinite ratio, which no model can use.

Missing Values as Features



GAPS ON EACH ROW

```
gaps = X_train.isna().sum(axis=1)

Companies with a gap: 2248  Most on one row: 32
```

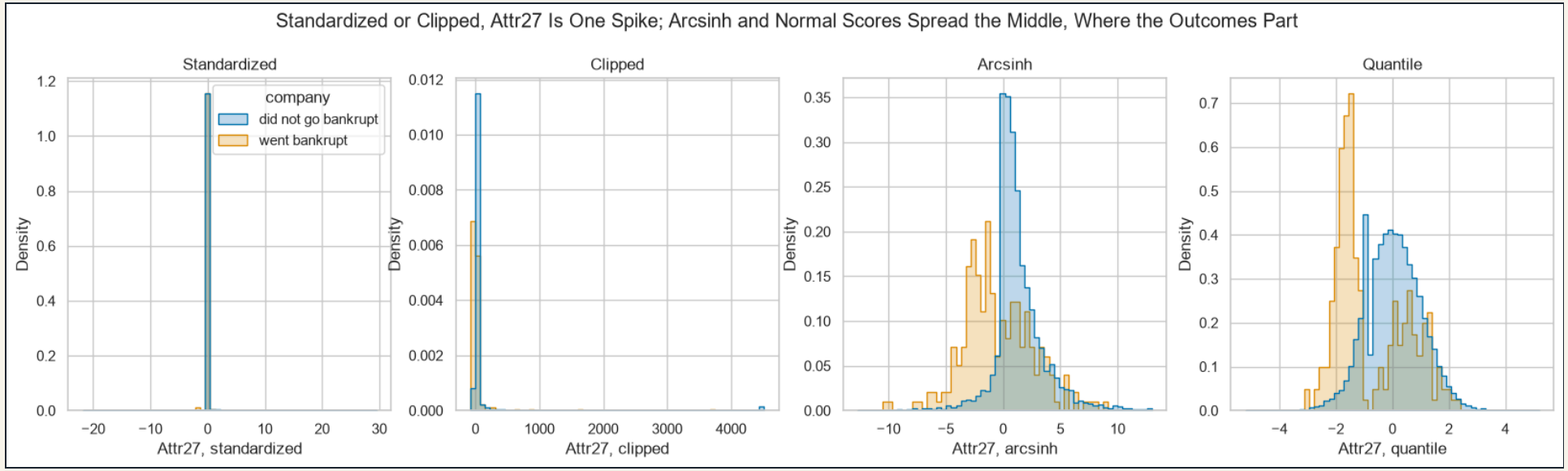
THE QUESTION

Q. Day 11 gave LightGBM the gaps as they are. Put a median imputer, without Course 4's missing indicators, in front of it. Write down the fold average precision, against 0.7783.

With no gap 3.6 percent went bankrupt; with two, 26.1 percent.

Reshape the coverage ratio four ways, then score.

Transforms for the Linear Model



ATTR27 AS IT COMES

```
Gaps: 317  Min: -158130.0  Median: 0.99  Max: 214880.0  Skew: 15.5
Median, went bankrupt: -1.29  did not: 1.02
```

THE QUESTION

Q. Course 4's logistic pipeline with **`StandardScaler`** in place of the quantile step, on all 64 ratios. Write down its fold average precision, against 0.6624.

Arcsinh and the quantile step spread the middle, where the two outcomes can be told apart.

Learned Inside the Fold

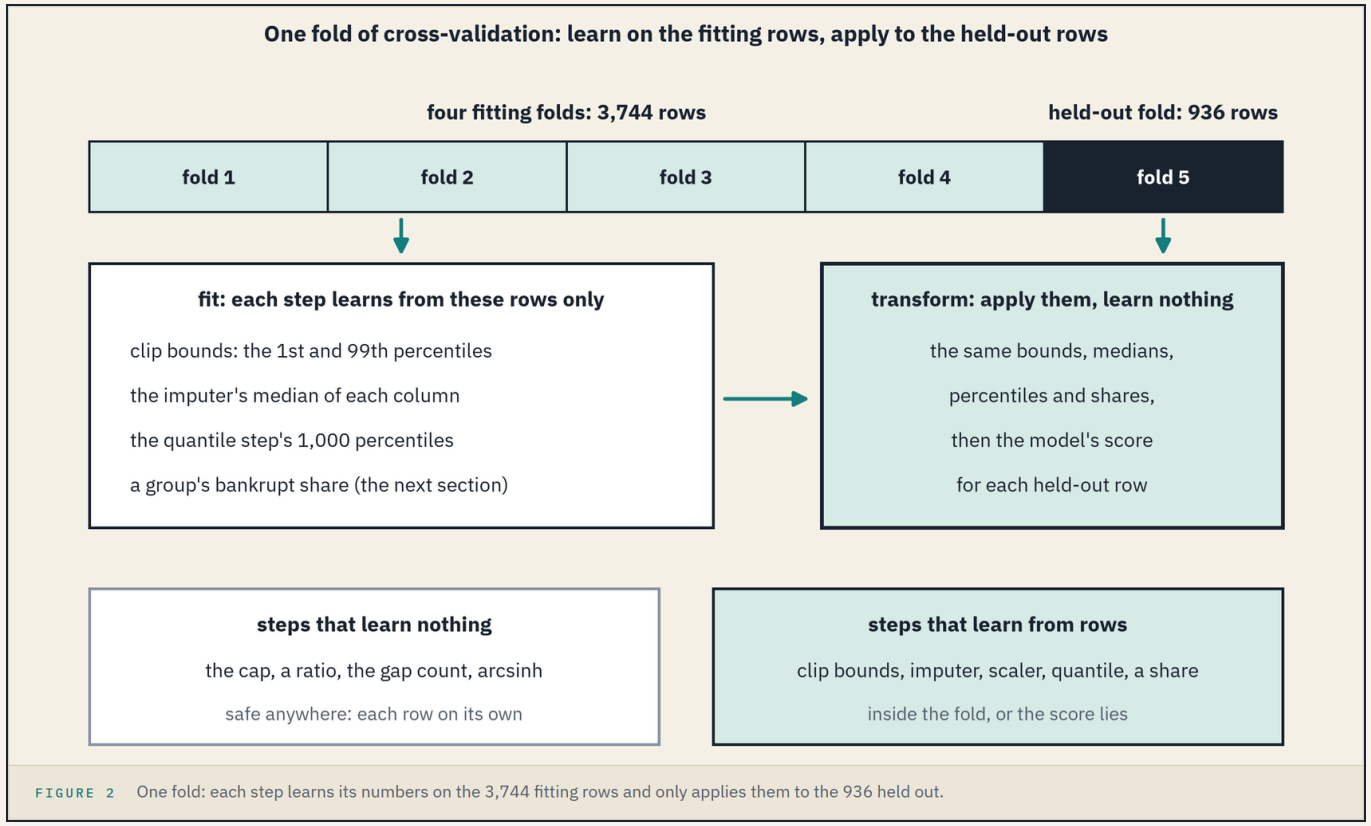


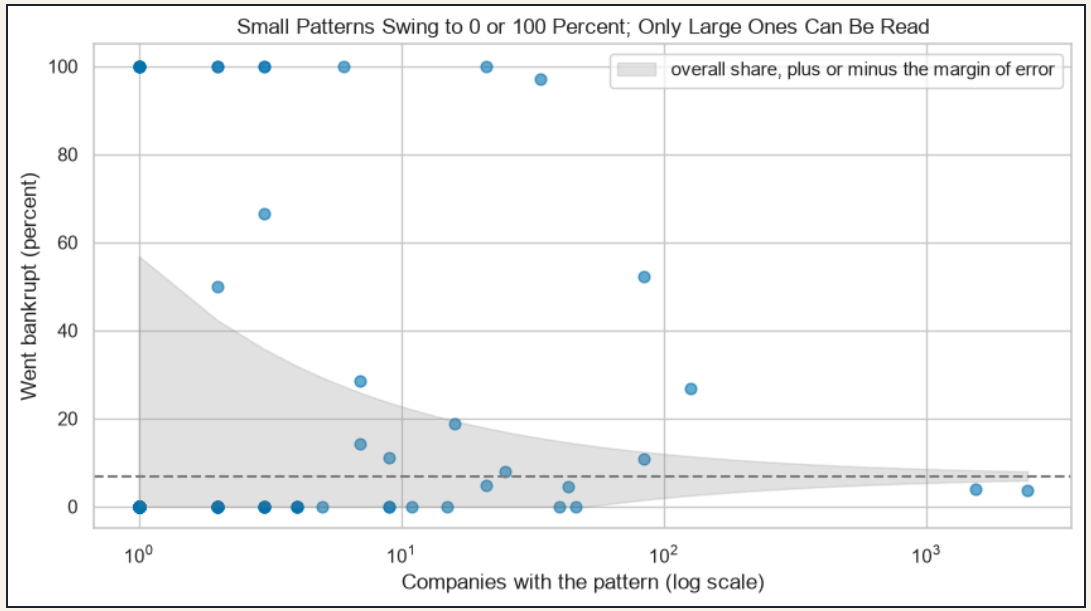
FIGURE 2 One fold: each step learns its numbers on the 3,744 fitting rows and only applies them to the 936 held out.

EACH FOLD'S CLIP BOUNDS FOR ATTR27

	LOW	HIGH
1	-71.0499	5345.6420
2	-73.4695	4243.1570
3	-70.5788	5225.1740
4	-138.9016	3832.2680
5	-71.0727	4873.0280

Each fold's bounds clip 14 to 21 of its 936 held-out rows, rows that had no say in those bounds.

A Feature Built From the Outcome



TARGET ENCODING

Target encoding replaces each group with its outcome share.

`TargetEncoder(target_type="binary", cv=...)` learns the shares at `fit` from the rows it is given, and pulls a small group's share toward the overall share.

EACH COMPANY'S GAP PATTERN

```
pattern = X_train.isna().astype(int).astype(str).agg("".join, axis=1)
groups = y_train.groupby(pattern).agg(["size", "mean"])
```

50 of the 52 patterns of 3 companies or fewer show exactly 0 or 100 percent.

Model Performance Comparison

THE QUESTION

Q. The three ratios, **`ratio_step`**, in front of LightGBM. Write down its fold average precision, against 0.7783.

THE RATIO STEP IN FRONT OF EACH MODEL

```
scores["LightGBM, ratios"] = fold_ap(Pipeline([("ratios", ratio_step), ("lightgbm", booster)]))
scores["logistic, ratios"] = fold_ap(Pipeline([("ratios", ratio_step), ("logistic", logistic)]))
summary(["LightGBM", "LightGBM, ratios", "logistic", "logistic, ratios"])
```

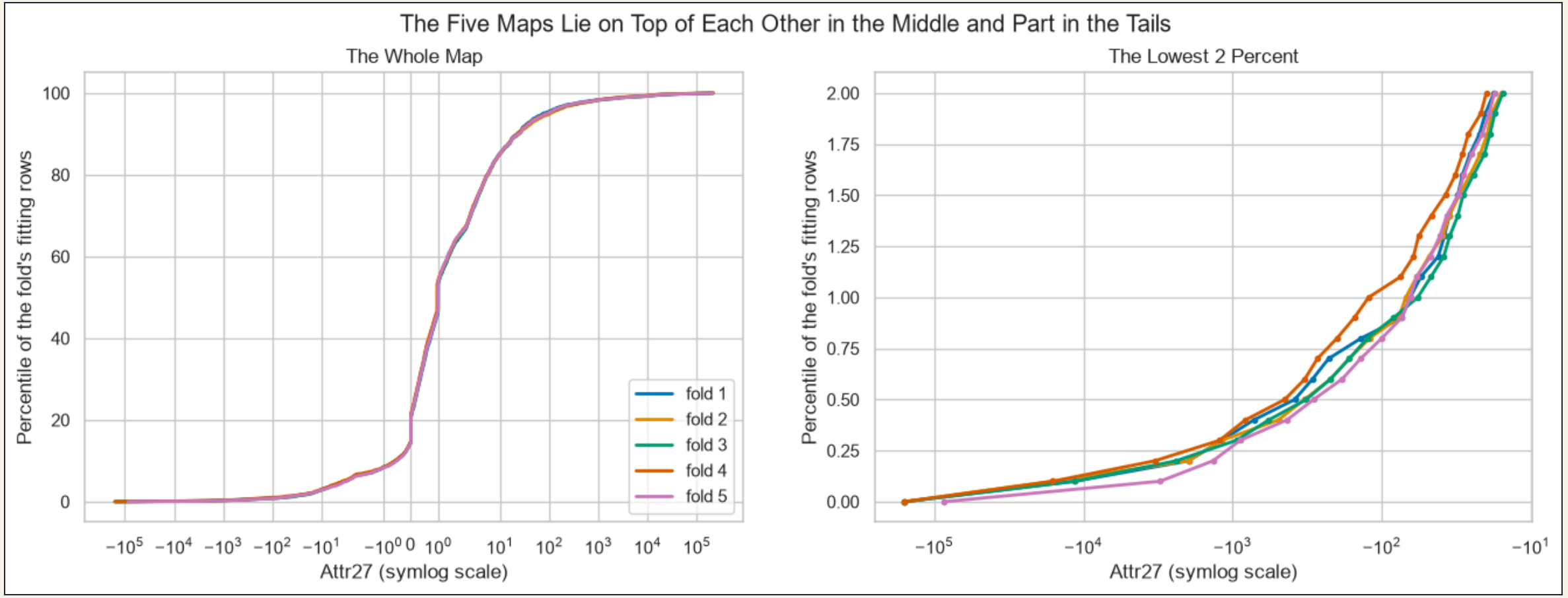
EACH CHANGE AGAINST ITS BASE MODEL

```
changes = pd.DataFrame({name: scores[name] - scores[name.split(",")[0]] for name in scores if "," in name})
changes.mean().sort_values()
```

A TORNADO FOR EACH BASE MODEL

```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
for ax, base in zip(axes, ["LightGBM", "logistic"]):
    names = [name for name in changes.mean().sort_values().index if name.startswith(base + ",")]
    sd = scores[base].std(ddof=0)
```

Insights and Conclusion



Day 13 keeps both models and rebalances the rows instead of the columns.

You Can Now

- Write a known shape into a column with a cap and score it on folds.
- Build a ratio from two ratios and guard its division with `.replace()`.
- Put a function in a pipeline with **`FunctionTransformer`**.
- Count gaps, and compare a median fill with LightGBM's own handling of gaps.
- Transform a long-tailed ratio for a linear model, and read what each fold's step learned.
- Encode a group's outcome share with **`TargetEncoder`** inside the folds.
- Weigh a new column's gain against the fold noise.

NEXT
WEEK 3 · DAY 3

Resampling, SMOTE, and Class Weights

Day 13 keeps both models and rebalances the rows instead of the columns.